

Performance Testing

Date	1 July, 2026
Team ID	SWTID-2026-8797
Project Na me	Credit Card Approv al P rediction
Maximum Marks	5 Marks

S.No	Metri c	Target Valu e	Actual Valu e	Status (pass/fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.28 sec	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	3.63 sec	Pass	Within acceptable li m i t
3	Throughput (Req/sec)	>40 req/sec	52 req/sec	Pass	Good request handling capacit y
4	Error Rate	<1%	0%	Pass	No request failures
5	CP U Utili zation	<80%	61%	Pass	Efficient CP U usage

6	Memory Utili zation	<80%	57%	Pass	Stable memory consumption
---	---------------------	------	-----	------	---------------------------

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JM eter, Postm an
Type of Testi ng	Load Testi ng, S tress Test ing
Target Module	Flask Prediction API (/pr edict), User Input F orm
Test Environment	Local S ystem (Windows 11, Python 3.11, Flask)
Test Date	1 July, 2026

Step 2: Test Scenarios

S.No	Test Scenario / Descrip tion	No. of virtual Users	Duration (sec)	Exp ected Ou tcome
1	Singl e user submits credit card application	1	30	Prediction generated successfully withi n 2 seconds
2	Conti nuous prediction requests	100	120	Stable response time with no crashes
3	Peak load testing	200	180	Application remains responsive and maintains accept able performan ce

Step 3: Performance Tes t Results

Step 4: Observations & Analysis

Key Fin d in gs

- The Credit C ard Approval P rediction system successfully processed con current user requests.
- Average p rediction respo nse time remained b elow two seconds.
- No fail ed pr ediction requ ests were obs erved du ring testing.
- Flask application remain ed stable under mod erate and heavy wo rkloads.
- Machine learning model deli vered accu rate pr edic tions without performanc e degradati on.

Bottlenecks Id entified

- Minor increas e in respon se time when more than 200 concurrent users acc essed the application.
- Initial model l oading cau sed a sli ght delay during the first prediction requ e st.

Optimization S teps Taken

- Optim ized Flask routing and request h andling.
- Reduced unnecessary preprocessing operati ons.
- Loaded the tr ained ML model once during application startup.
- Enabled efficient NumP y operati ons for faster p re dictions.
- Improved input validation to reduce p rocessing ov erhead.

Step 5: Screenshots / Evidence

